SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project: CareerConnect — ICTAK Capstone Internship & Job Portal

Compliant with IEEE Std 830-1998 Standard

Prepared by: Fayiz Mohammed (`fayiz225027@sahrdaya.ac.in`)
Institution: Sahrdaya College of Engineering and Technology
Program: Full Stack Application Development with ReactJS (MERN Stack)
Presented to: Information & Communication Technology Academy of Kerala (ICTAK)
Date: October 25, 2026
Status: Approved / Release 1.0

TABLE OF CONTENTS

1. Introduction
2. 1.1 Purpose
3. 1.2 Document Conventions
4. 1.3 Intended Audience and Reading Suggestions
5. 1.4 Product Scope
6. 1.5 References
7. Overall Description
8. 2.1 Product Perspective
9. 2.2 Product Functions
10. 2.3 User Classes and Characteristics
11. 2.4 Operating Environment
12. 2.5 Design and Implementation Constraints
13. 2.6 User Documentation
14. 2.7 Assumptions and Dependencies
15. External Interface Requirements
16. 3.1 User Interfaces
17. 3.2 Hardware Interfaces
18. 3.3 Software Interfaces
19. 3.4 Communications Interfaces
20. System Features & Functional Requirements

21. 4.1 Authentication & Role-Based Access Control
22. 4.2 Job & Internship Catalog Management
23. 4.3 Search, Filtering & Categorization Engine
24. 4.4 Candidate Application Lifecycle
25. 4.5 Recruiter Evaluation & Feedback Management
26. 4.6 Bookmarks & Profile Configuration
27. 4.7 Academic Administration & Oversight
28. Other Non-functional Requirements
29. 5.1 Performance Requirements
30. 5.2 Safety and Security Requirements
31. 5.3 Software Quality Attributes
32. Data Model & Database Schema
33. Acceptance Criteria & Verification Matrix

---

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional, non-functional, and structural requirements for **CareerConnect**, a MERN Stack web application developed as the final Capstone Project for the ICT Academy of Kerala (ICTAK) Full Stack Development curriculum. It defines system behavior, interfaces, constraints, and validation criteria for all project stakeholders.

1.2 Document Conventions

- Requirements are prioritized using MoSCoW methodology (Must have, Should have, Could have).
- REST API contracts specify HTTP methods (`GET` , `POST` , `PUT` , `DELETE`) and standard RFC status codes.
- Database models follow Mongoose/MongoDB document notations.

1.3 Intended Audience and Reading Suggestions

- **Academic Evaluators & Mentors (ICTAK):** Review sections 2, 4, 6, and 7 to verify capstone compliance.
- **Software Developers & Maintainers:** Review sections 3, 4, and 6 for architecture, data models, and API endpoints.
- **End-Users (Students, Recruiters):** Review section 2.2 and 2.3 for user journeys and feature scope.

1.4 Product Scope

CareerConnect addresses the gap in modern campus placement and internship workflows. The software provides an end-to-end bridge between aspiring students seeking software engineering internships and

tech recruiters seeking verified junior talent. The system encompasses user authentication, role-based dashboards, opportunities cataloging, keyword search, multi-factor filtering, resume link submissions, applicant status tracking, and central moderation.

1.5 References

1. IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).
2. ICTAK MERN Stack Project Guidelines & Student Reference Guide (2026).
3. React Official Documentation: https://react.dev/
4. Node.js & Express Documentation: https://expressjs.com/
5. MongoDB Atlas Documentation: https://www.mongodb.com/docs/atlas/

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

CareerConnect is an autonomous, cloud-hosted web application built on the MERN stack: - **Client Tier:** Single Page Application (SPA) built using React 18 and Vite. - **Application Server Tier:** RESTful API service built using Express 4.x running on Node.js runtime. - **Data Persistence Tier:** Cloud-managed NoSQL document store using MongoDB Atlas with Mongoose 8.x ODM.

2.2 Product Functions

1. **User Authentication:** Registration and login with salted bcrypt hashing and JWT token issuance.
2. **Role-Based Workflows:** Dynamic navigation and protected access for `student`, `recruiter`, and `admin`.
3. **Opportunity Management:** Creation, editing, status toggling, and deletion of job and internship listings.
4. **Search & Multi-Filter Engine:** Instant filtering by domain category, job type, location, and keywords.
5. **Application Lifecycle:** Student resume submission, duplicate prevention, and real-time status updates (`Pending`, `Reviewed`, `Shortlisted`, `Accepted`, `Rejected`).
6. **Recruiter Evaluation:** Candidate inspection, cover note review, and recruiter feedback messaging.
7. **Platform Administration:** Global metrics, active user auditing, and listing moderation.

2.3 User Classes and Characteristics

User Class	Description	Technical Literacy	Frequency of Use
Student	Enrolled candidates looking for internships or graduate positions.	Intermediate	Daily / Weekly
Recruiter	Talent partners and software company HR representatives posting openings.	Intermediate	Weekly
Administrator	Academic coordinators and system moderators oversight.	High	Administrative oversight

2.4 Operating Environment

- **Browser Compatibility:** Google Chrome (v100+), Mozilla Firefox (v100+), Microsoft Edge (v100+), Safari (v15+), mobile viewports.
- **Server Environment:** Node.js v18.x to v24.x LTS.
- **Database:** MongoDB 6.0+ Cluster (MongoDB Atlas Cloud).

2.5 Design and Implementation Constraints

- **Separation of Concerns:** Client must never query MongoDB directly; all data transfer happens over secure JSON REST endpoints.
- **Stateless Authentication:** Server maintains no session state; security is enforced via signed JWT bearer tokens.
- **Zero Exposure of Secrets:** All API keys, database credentials, and secrets reside in server-side environment variables (`.env`), never exposed to frontend code.

---

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

- **Responsive Layout:** Responsive CSS Grid and Flexbox layouts supporting desktop (1440px), tablet (768px), and mobile viewports (375px).
- **Consistent Visual Design:** Clean typography (Inter, Plus Jakarta Sans), semantic state badges (`success` , `warning` , `danger` , `info`), and accessible contrast ratios (WCAG 2.1 AA compliant).
- **Four Screen States:** All dynamic views implement distinct Loading (animated spinners), Success, Empty (informative placeholders), and Error states.

3.2 Software Interfaces

- **Express Backend API:** Exposes RESTful endpoints at `/api/*` .

- **Axios HTTP Client:** Integrated with request interceptor for JWT authorization header injection and response interceptor for token expiration handling.
- **Mongoose ODM:** Defines rigid schemas, type coercion, validations, and indexes.

---

4. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS

4.1 Authentication & Role-Based Access Control

- **FR-AUTH-1:** The system shall allow users to register with name, email, password, role (`student` or `recruiter`), company (if recruiter), location, and phone number.
- **FR-AUTH-2:** Passwords must be hashed using `bcryptjs` with a minimum salt factor of 10 prior to persistence.
- **FR-AUTH-3:** Upon successful authentication, the server shall issue a signed JWT containing user ID and expiration duration (7 days).
- **FR-AUTH-4:** Client SPA shall guard protected routes and prevent unauthorized role transitions using `ProtectedRoute` component.

4.2 Job & Internship Catalog Management

- **FR-JOB-1:** Authorized recruiters and admins shall be able to create new opportunities specifying title, company, location, employment type, domain category, salary, experience requirement, deadline, description, requirements, and responsibilities.
- **FR-JOB-2:** Recruiters shall only have permission to update or delete postings that were authored by their own account.
- **FR-JOB-3:** Deleting a job shall cascade delete all associated candidate applications.
- **FR-JOB-4:** Recruiters can toggle opportunity status between `Active` and `Closed` .

4.3 Search, Filtering & Categorization Engine

- **FR-SEARCH-1:** The system shall provide full-text keyword search spanning job titles, companies, and descriptions.
- **FR-SEARCH-2:** Users shall be able to filter listings by Category (`Full Stack` , `Frontend` , `Backend` , `Data Science` , `DevOps` , `UI/UX` , `Cybersecurity`).
- **FR-SEARCH-3:** Results shall be paginated with configurable page sizes (default: 9 records per page).

4.4 Candidate Application Lifecycle

- **FR-APP-1:** Logged-in students can apply to any active opportunity by providing a resume/portfolio URL, experience level, and optional cover letter.
- **FR-APP-2:** The system shall reject duplicate applications for the same position by the same student using a compound unique database index (`job + applicant`).
- **FR-APP-3:** Students can view the live progress of all submitted applications on the `My Applications` page.

- **FR-APP-4:** Students can withdraw any application that remains in `Pending` status.

4.5 Recruiter Evaluation & Feedback Management

- **FR-EVAL-1:** Recruiters can review all candidates for their posted jobs in a dedicated evaluation dashboard.
- **FR-EVAL-2:** Recruiters can transition candidate status between `Reviewed`, `Shortlisted`, `Accepted`, and `Rejected`.
- **FR-EVAL-3:** Recruiters can provide constructive evaluation feedback messages that display directly on the candidate's dashboard.

4.6 Bookmarking & Profile Configuration

- **FR-PROF-1:** Students can bookmark opportunities to save them to their personalized dashboard.
- **FR-PROF-2:** Users can update their bio, skills, resume link, portfolio link, and contact details.

4.7 Academic Administration & Oversight

- **FR-ADM-1:** Administrators have complete platform analytics including total user counts, active listings, and total application throughput.
- **FR-ADM-2:** Administrators can moderate users, delete fraudulent postings, and view user registries.

---

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

- **Response Time:** API endpoints shall respond in under 300ms under typical operating conditions.
- **Asset Optimization:** Client production bundle compiled via Vite shall not exceed 350kB gzipped.

5.2 Security Requirements

- **No Plaintext Credentials:** Passwords are never returned in API payloads (`select: false` on Mongoose schema).
- **CORS Protection:** Cross-Origin Resource Sharing is strictly constrained to authorized origins.
- **Injection Prevention:** Parameterized queries and Mongoose schema types prevent NoSQL operator injection.

5.3 Reliability & Availability

- Hosted on highly available cloud infrastructure (MongoDB Atlas multi-region cluster and containerized server execution).

---

6. DATA MODEL & DATABASE SCHEMA

6.1 Users Collection (`User`)

- `_id` : ObjectId (Primary Key)
- `name` : String (Required, max 60 chars)
- `email` : String (Required, Unique, Lowercase, Regex validated)
- `password` : String (Required, Bcrypt hashed, select: false)
- `role` : String (Enum: `['student', 'recruiter', 'admin']` , Default: `'student'`)
- `company` : String
- `location` : String
- `phone` : String
- `bio` : String
- `skills` : [String]
- `portfolio` : String
- `resumeUrl` : String
- `savedJobs` : [ObjectId -> ref: Job]
- `createdAt` , `updatedAt` : Date (Automatic Timestamps)

6.2 Jobs Collection (`Job`)

- `_id` : ObjectId (Primary Key)
- `title` : String (Required, max 100 chars)
- `company` : String (Required)
- `location` : String (Required)
- `jobType` : String (Enum: `['Internship', 'Full-time', 'Part-time', 'Contract', 'Remote']`)
- `category` : String (Enum: `['Full Stack (MERN)', 'Frontend / React', 'Backend / Node.js', 'Data Science & AI', 'Cloud & DevOps', 'UI/UX Design', 'Cybersecurity']`)
- `description` : String (Required)
- `requirements` : [String]
- `responsibilities` : [String]
- `salary` : String (Default: 'Best in Industry')
- `experienceLevel` : String (Enum: `['Internship / Fresher', 'Entry Level (0-1 yr)', 'Mid Level (1-3 yrs)', 'Senior (3+ yrs)']`)
- `deadline` : Date
- `postedBy` : ObjectId -> ref: User (Required)
- `status` : String (Enum: `['Active', 'Closed']` , Default: `'Active'`)
- `applicantsCount` : Number (Default: 0)

- `createdAt`, `updatedAt`: Date

6.3 Applications Collection (`Application`)

- `_id`: ObjectId (Primary Key)
- `job`: ObjectId -> ref: Job (Required)
- `applicant`: ObjectId -> ref: User (Required)
- `coverLetter`: String
- `resumeUrl`: String (Required)
- `experienceYears`: String (Default: 'Fresher')
- `status`: String (Enum: `['Pending', 'Reviewed', 'Shortlisted', 'Accepted', 'Rejected']`, Default: `'Pending'`)
- `recruiterFeedback`: String
- `createdAt`, `updatedAt`: Date
- **Indexes:** Compound unique index `{ job: 1, applicant: 1 }`

---

7. ACCEPTANCE CRITERIA & VERIFICATION MATRIX

Requirement ID	Acceptance Test	Verification Result
FR-AUTH-1	Register new user with student role; verify database entry and hashed password.	**PASS**
FR-AUTH-3	Login with valid credentials; verify JWT issued and stored in browser storage.	**PASS**
FR-JOB-1	Recruiter posts job with title, stipend, and description; verify job displays in public catalog.	**PASS**
FR-SEARCH-1	Filter catalog by keyword "React"; verify only matching postings render.	**PASS**
FR-APP-1	Student applies to opportunity; verify applicant counter increments and application displays in My Applications.	**PASS**
FR-APP-2	Student attempts to apply again to the same position; verify HTTP 409 Conflict with friendly alert.	**PASS**
FR-EVAL-2	Recruiter changes candidate status to "Shortlisted"; verify change propagates to student view.	**PASS**
FR-ADM-1	Admin logs in; verify platform-wide user and job statistics calculate correctly.	**PASS**